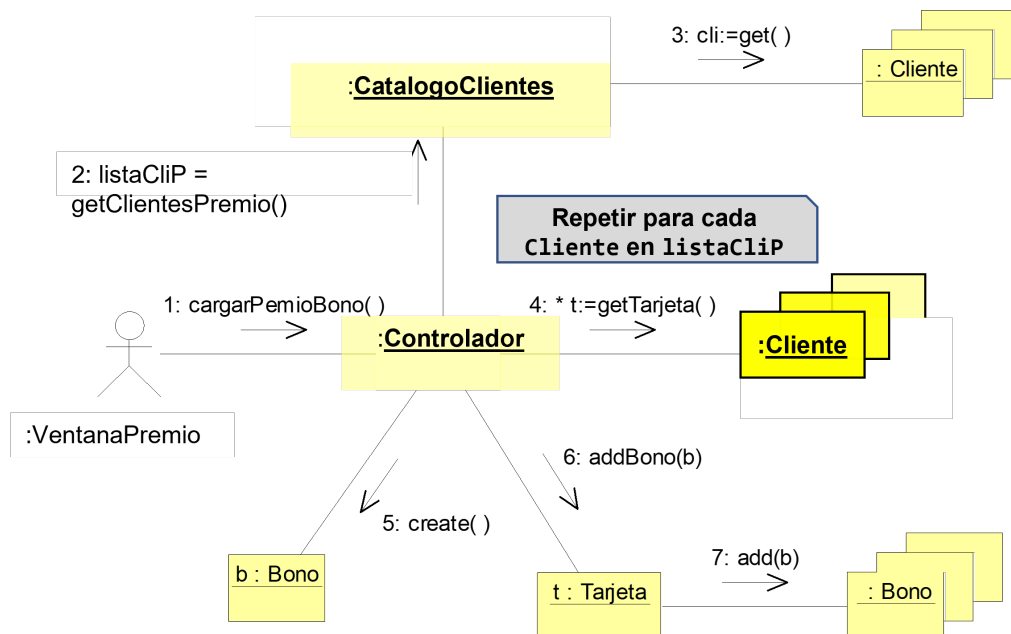


Boletín ejercicios patrones GRASP

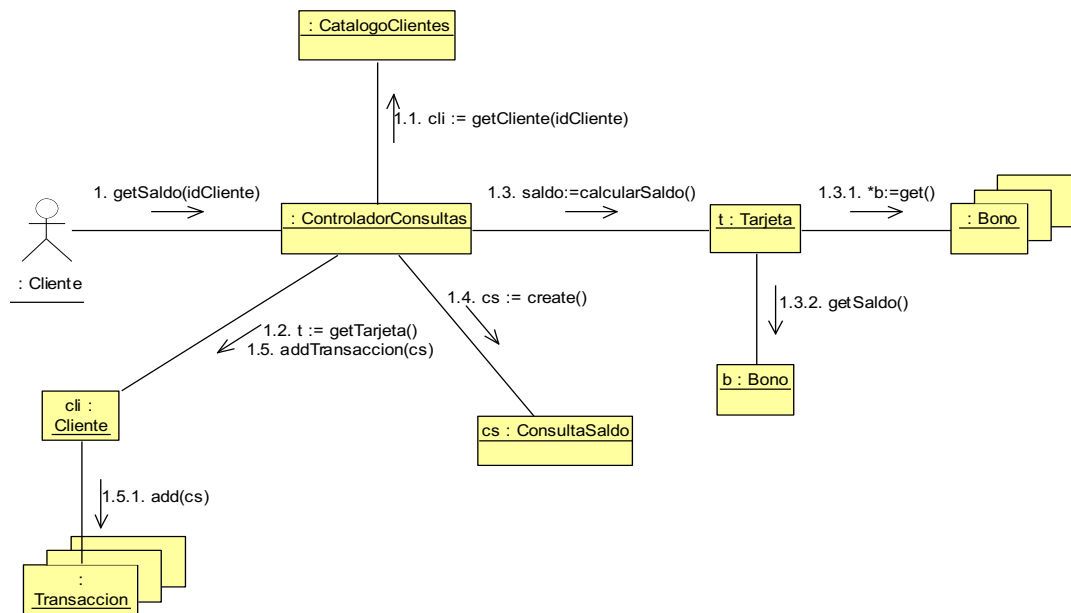
Los patrones GRASP son un conjunto de principios básicos de programación orientada a objetos que establecen cómo asignar responsabilidades a las clases y entre los que se encuentran los patrones **Experto**, **Creador** y **Controlador**. El patrón *Experto* establece la asignación de una responsabilidad a la clase que tenga la información necesaria para cumplimentarla; el patrón *Creador* establece que las instancias de cierta clase deben ser creadas en otra clase que las registre, almacene en alguna estructura de datos o tenga información de inicialización; y el patrón *Controlador* está relacionado con quién gestiona los eventos de entrada a un sistema, estableciendo que deben ser manejados por una clase que separe la capa de negocio de la capa de presentación.

En los dos primeros ejercicios, se expresa cómo se implementa una determinada funcionalidad o responsabilidad mediante una interacción de objetos de diferentes clases, y se debe indicar si se viola alguno de los patrones GRASP mencionados arriba y modifique la colaboración de objetos para que no se viole ninguno. En algunos de ellos también se pide que se escriba el código antes y después de la corrección. Los últimos tres ejercicios son preguntas de exámenes de TDS de cursos anteriores.

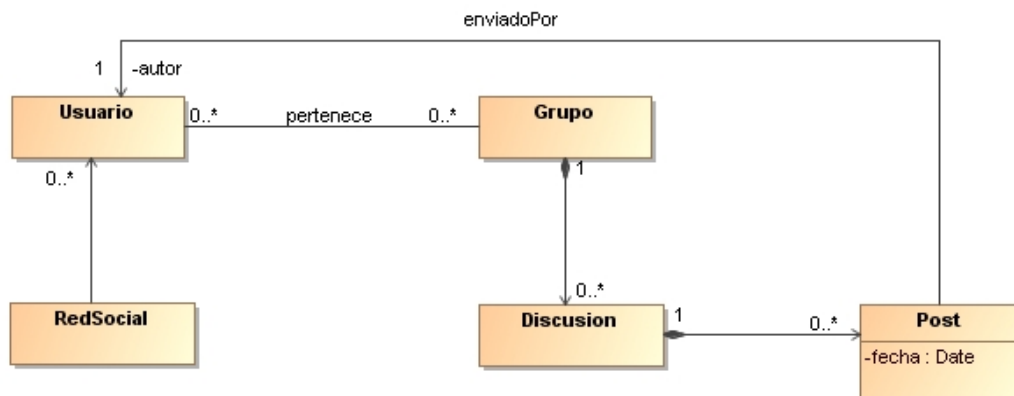
1. El siguiente diagrama UML muestra una posible interacción relacionada con un caso de uso “Regalo Bono 3G” de una aplicación para una operadora de telefonía móvil que ofrece tarjetas 3G de prepago para conexión a Internet. La funcionalidad de la interacción es la siguiente: cuando compras un bono para cierto número de horas de conexión (o de megabytes transferidos) existe la posibilidad de conseguir otro bono de regalo según un sorteo que se realiza a final de mes. Desde una ventana se lanza la carga del bono regalo a la tarjeta de los clientes premiados. Un objeto *Cliente* tiene una referencia a su objeto *Tarjeta* que puede tener asociados uno o más objetos *Bono*.



2. Dado el siguiente diagrama de comunicación UML que muestra una posible interacción para la consulta de saldo por parte de un cliente de una operadora de telefonía móvil que ofrece tarjetas 3G de prepago para conexión a Internet, **señala qué patrones GRASP son violados y modifica la interacción de modo que no se violen dichos patrones**. La funcionalidad de la interacción es la siguiente: un cliente realiza una petición de consulta de saldo y el sistema calcula el saldo como la suma del saldo de cada bono cargado en la tarjeta, además crea una transacción de tipo “consulta de saldo” que la registra en el cliente. Un objeto *Cliente* tiene una referencia a su objeto *Tarjeta* que puede tener asociados uno o más objetos *Bono*; cada cliente tiene una colección de objetos *Transacción* en la que se almacenan objetos transacción como *ConsultaSaldo*.



3. El diagrama de clases de abajo muestra una parte del modelo de dominio de una aplicación que gestiona la red social “Indignados del Mundo”. Un usuario puede pertenecer a alguno de los grupos existentes. En cada grupo se inician discusiones a las que los usuarios pueden añadir posts. Cada post registra la fecha y el usuario que lo envió. La aplicación ofrece la funcionalidad de calcular el número de posts enviados por un usuario en cierto intervalo de tiempo a través del método `numPostsUsuarioEnPeriodo()` que es mostrado abajo y que está incluido en la clase `RedSocial` que actúa como controlador. Indica si se viola algún patrón GRASP. Justifica la respuesta utilizando un diagrama de secuencia.




```

        new SimpleDateFormat("dd-MM-yyyy");
    try {
        Date fechaPub = formatter.parse(pub.getFecha());
        if (fechaPub.after(date1) && fechaPub.before(date2)) {
            textArea.append(pub.getTitulo()+"\n");
        }
    } catch (ParseException e1) {
        e1.printStackTrace();
    }
}
});
panel.add(btnVerPublicaciones);
JScrollPane scrollPane = new JScrollPane();
panel_1.add(scrollPane);
textArea = new JTextArea();
scrollPane.setViewportViewView(textArea);
}
}

```

5. El siguiente código es parte de una aplicación bancaria que dispone de la funcionalidad de *bloquear todas las cuentas de un cliente* a petición de Hacienda. La clase `ViewGestionCuentas` implementa una ventana a través de la cual se pueden realizar diversas operaciones sobre las cuentas de un cliente. La clase `Cliente` tiene un atributo `cuentas` de tipo `ArrayList<Cuenta>` que registra todas las cuentas de ese cliente y el método `getCuentas()` retorna esa colección. La clase `Cuenta` tiene un atributo `bloqueada` de tipo booleano que determina si una cuenta está bloqueada y el método `setBloqueada()` que establece su valor. Al bloquear la cuenta se crea una instancia de la clase `Bloqueo` que es una subclase de la clase abstracta `Transaccion`. Esta clase tiene un atributo para registrar la fecha actual y sus subclases añaden atributos según el tipo de transacción. La clase `Cuenta` también tiene un atributo `transacciones` de tipo `ArrayList<Transaccion>` que registra todas las transacciones realizadas sobre una cuenta y el método `getTransacciones()` que retorna dicha colección. La clase `CatálogoClientes` es una clase que implementa un catálogo de clientes.

```

public class ViewGestionCuentas extends JFrame {
    private Cliente cliente;
    private JTextField campoDNI;
    ...
    public ViewGestionCuentas() {
        ...
        JPanel panel = new JPanel();
        JLabel rotuloDNI = new JLabel("DNI Cliente ");
        campoDNI = new JTextField();
        campoDNI.setText("0");
        panel.add(rotuloDNI);
        panel.add(campoDNI);
        ...
        JButton botonBloquear = new JButton("Bloquear");
        botonBloquear.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ev) {
                Cliente cliente = CatalogoClientes.getInstancia().
                    getCiente(campoDNI.getText());

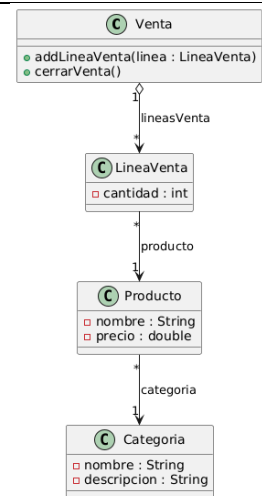
                if (cliente != null) {
                    ArrayList<Cuenta> cuentas = cliente.getCuentas();
                    for (Cuenta cuenta: cuentas) {
                        cuenta.setBloqueada(true);
                        ArrayList<Transaccion> transacciones =
                            cuenta.getTransacciones();
                        transacciones.add(new Bloqueo(new Date()));
                    }
                } else JOptionPane.showMessageDialog(ViewGestionCuentas.this,
                    "No existe cliente con ese dni");
            }
        });
        ...
    }
}

```

6. Sea una aplicación de una **plataforma de compras online**. El cliente puede iniciar una nueva venta, realizar búsquedas de productos o clicar sobre uno de los productos devueltos para ver información sobre él, como su nombre, precio y precio medio de venta. La interfaz de usuario también muestra un botón “Añadir a la venta”

para que el cliente pueda añadir el producto seleccionado a la venta actual, indicando la **cantidad** del producto que se desea comprar (añadir a la venta).

El diagrama de clases UML muestra las **clases del dominio** de la aplicación de venta involucradas en este ejercicio (no se muestran todos los atributos, pero sí los necesarios). Un objeto **Producto** tiene campos para registrar su nombre y precio. Aunque no aparezca en el diagrama, el sistema ofrece un repositorio de productos llamado **RepoProductos**. Un objeto **Venta** registra una lista de objetos **LineaVenta** que representan los productos incluidos en la venta actual. Un objeto **LineaVenta** referencia al producto vendido y la cantidad seleccionada por el cliente. Un objeto **Categoria** tiene un campo para registrar el nombre de la categoría a la que pertenece el producto y otro para su descripción.



Abajo se muestra un código JavaFX de una clase ventana que corresponde a la interfaz descrita anteriormente.

```

public class VentanaMostrarProductoFX extends Application {

    private Venta ventaActual; // venta en curso
    private List<Producto> productos; // productos devueltos por la búsqueda
    private Producto productoActual; // producto actualmente seleccionado

    @Override
    public void start(Stage stage) {

        ventaActual = new Venta();

        TextField txtProducto = new TextField();
        txtProducto.setPromptText("Nombre del producto");
        Button btnBuscarProducto = new Button("Buscar producto");
        btnBuscarProducto.setOnAction(ev -> {
            productos = buscarProductos(txtProducto.getText());
        });

        TextField txtCantidad = new TextField();
        txtCantidad.setPromptText("Cantidad");
        Button btnAñadirLinea = new Button("Añadir a la venta");
        btnAñadirLinea.setOnAction(ev -> {
            nuevaLineaVenta(productoActual,
                            Integer.parseInt(txtCantidad.getText()));
        });

        Button btnFinalizarVenta = new Button("Finalizar venta");
        btnFinalizarVenta.setOnAction(ev -> finalizarVenta());
        ...
        stage.show();
    }

    private List<Producto> buscarProductos(String txtBusqueda) {
        List<Producto> resultados = new ArrayList<>();
        List<Producto> productos = RepoProductos.INSTANCE.getTodosProductos();

        for (Producto producto : productos) {
            if (producto.getNombre().contains(txtBusqueda)) {
                resultados.add(producto);
            }
        }
        return resultados;
    }

    private void nuevaLineaVenta(Producto producto, int cantidad) {
        LineaVenta linea = new LineaVenta(producto, cantidad);
        ventaActual.addLineaVenta(linea);
    }

    private void finalizarVenta() {
        ventaActual.cerrarVenta();
        ...
    }
}
  
```

1. Indica qué **patrones GRASP** se han violado en el código anterior. Justifica tu respuesta de forma **muy breve y clara**. Indica también por qué se ha violado el principio de **separación modelo–vista**.
2. Reescribe el código anterior de forma que **no se viole ningún patrón GRASP ni el principio de separación modelo-vista**. Si al reescribirlo se definen nuevos métodos o clases, se debe escribir su código e indicar **la clase en la que se añade**.